CHAPTER SIX

Working with AI

"The future belongs to those who learn more skills and combine them in creative ways."

— ROBERT GREENE

In an academic setting, the consequences of shallow learning or synthetic competence are largely contained. You fail an exam, struggle through a lab, or realize during a seminar that you spent an afternoon reading AI summaries instead of building real intuition.

In the workplace, the stakes change fundamentally.

Professional work does not exist in an abstract vacuum. It interfaces directly with production database clusters, financial ledgers, patient healthcare workflows, enterprise security perimeters, and critical infrastructure. When you push a pull request to a shared repository, submit an architectural proposal, or draft an engineering specification, other human beings rely on your artifact to make decisions, allocate capital, and build subsequent layers of the system.

Generative artificial intelligence has introduced an intoxicating surge of velocity to the modern workplace. A boilerplate API handler that once required thirty minutes of repetitive typing can be generated in three seconds. A complex SQL migration script appears in a single prompt. For companies and individual contributors measured on raw deliverable volume, this feels like an unalloyed superpower.

Yet beneath this surge of speed lies a critical professional dilemma: throughput is not competence. The defining challenge of the modern workplace is no longer how quickly we can flood repositories and inboxes with generated artifacts. It is whether we truly understand, control, and hold ourselves personally accountable for what we release into the world.

Speed Without Skill

The earliest sensations of integrating generative tools into daily professional engineering are exhilaration and relief. The mental drag of writing repetitive boilerplate, scaffolding unit tests, and configuring deployment YAML files evaporates. Backlogs shrink, pull request throughput climbs, and engineering velocity dashboards trend sharply upward.

However, when organizations celebrate velocity without demanding deep comprehension, a subtle form of cognitive deskilling takes root. When an engineer routinely prompts a language model to generate distributed lock handlers or database index migrations without understanding locking semantics, connection pool exhaustion, or index cardinality, the immediate feature ships ahead of schedule. But the developer has not grown an inch.

In fact, the developer's diagnostic instincts actively wither through disuse. When you no longer trace execution paths by hand, wrestle with subtle type constraints, or visualize concurrency hazards in working

memory, your intuition for system behavior atrophies. You become an operator who is exceptionally skilled at requesting solutions from a model, yet completely helpless when the generated solution behaves unpredictably in production.

This vulnerability remains hidden during calm operational periods. It surfaces with brutal clarity during high-severity production outages. When a distributed microservice starts dropping transactions under heavy load at three o'clock on a Sunday morning, there is no time to engage in a leisurely, exploratory chat session with a model that lacks real-time telemetry, private network topology context, and operational history. In that crucible, the team desperately needs engineers who understand memory lifecycles, kernel socket buffers, and network serialization down to the bare metal.

Speed without foundational skill creates brittle software architectures: systems that are astonishingly fast to build, yet catastrophic to maintain and impossible to debug.

The most difficult dimension of professional engineering has never been the physical act of typing code. Typing accounts for less than ten percent of an engineer's true labor. The genuine work consists of problem disambiguation, discovering hidden edge cases, enforcing architectural boundaries, anticipating failure modes, and negotiating human trade-offs. When tools make generation frictionless, they tempt us to spend all our energy generating volume rather than sharpening our judgment about what is actually worth building.

Collaboration, Not Replacement

Much of the public debate surrounding artificial intelligence in the workplace swings wildly between two unhelpful dogmas. On one side are the techno-evangelists who announce that software engineers, analysts, and knowledge workers will soon be wholly automated away by autonomous agents. On the other side are the traditionalists who dismiss generative models as unreliable toys with no place in serious production workflows.

Both positions miss the practical reality of modern craftsmanship: the winning pattern is neither total abdication to the machine nor stubborn refusal to use modern tools. It is disciplined, adversarial collaboration.

To collaborate effectively with AI at work, you must maintain a clear, unromantic view of the division of labor. Large language models excel at statistical pattern matching across vast codebases, translating schemas between different programming languages, generating repetitive unit test scaffolding, and suggesting candidate implementations for well-defined algorithmic puzzles. They are tireless, hyper-literate assistants.

However, the model has zero skin in the game. It possesses no situational awareness of your company's unique operational culture, no empathy for the frustrated customer dealing with a broken interface, and no moral conscience when evaluating security risks or ethical trade-offs.

- **The Replacement Trap (Passivity):** The engineer surrenders agency, treating the AI as an autonomous oracle that takes a vague feature requirement and delivers a complete, unvetted pull request. The human becomes a glorified copy-paste conveyor belt. This path destroys personal leverage and turns the engineer into a replaceable operator.
- **The Collaborative Pattern (Mastery):** The engineer acts as the director, systems architect, and chief verifier. They use AI to explore candidate implementations, sketch alternative database schemas, and identify potential security blind spots. But the architectural vision, trade-off selections, and final verification remain entirely in human hands.

Offload the digital drudgery—reformatting legacy configuration files, generating repetitive test fixtures, and parsing obscure log formats—so that you can invest your finite cognitive energy into high-leverage architectural reasoning, mentoring teammates, and solving the hard domain problems that machines cannot see.

Teams and Trust

When an individual adopts AI in isolation, their workflow remains a personal choice. But when generative tools enter an engineering organization, they alter the social fabric of how teams review code, evaluate deliverables, and establish professional trust.

Without explicit team norms, engineering departments quickly descend into an environment of synthetic noise, where unverified AI outputs are passed from one person to another in an empty game of corporate theater.

Consider a dynamic that is becoming alarmingly prevalent in tech companies: an engineer uses an AI assistant to generate a four-hundred-line pull request in ten minutes without fully reading the implementation. A second engineer, assigned to perform the code review, uses an AI plugin to summarize the diff and generate generic comments. A product manager then uses a third tool to synthesize the pull request title and description for a weekly stakeholder report.

On the surface, metrics show stellar performance: high commit volume, rapid code review turnaround, and exhaustive release notes. Yet beneath the surface, not a single human mind has actually read the implementation, evaluated the architectural coupling, or verified whether the code introduces a catastrophic memory leak.

Trust inside a high-performing team is built on the shared certainty that when a colleague signs their name to a pull request, design doc, or post-mortem, they have applied their own critical discernment to every single line. If teammates begin to suspect that deliverables are simply unvetted machine output, professional trust disintegrates. Code review either becomes a rubber stamp that lets defects slip into production, or reviewers become paralyzed by adversarial paranoia.

Healthy engineering teams prevent this breakdown by enforcing three non-negotiable standards:

- **Radical Transparency:** Engineers openly disclose when and how generative tools were used to scaffold code or draft documentation. Transparency eliminates stigma and directs the team's review attention to machine-generated sections.
- **Heightened Scrutiny for Synthetic Code:** Reviewers must understand that AI bugs do not look like human bugs. Human errors are often clumsy or incomplete; AI errors are syntactically pristine, exquisitely formatted, highly confident, and structurally deceptive. Synthetic code requires more rigorous human review, not less.
- **Absolute Personal Liability:** The engineer who opens the pull request carries one hundred percent of the accountability for its correctness, performance, security, and maintainability. Saying "the AI generated it that way" is recognized as an unacceptable breach of professional duty.

Careers When Tools Change

Whenever a technological paradigm shifts, anxiety ripples through the industry. Headlines proclaim the death of junior developers, and people scramble to figure out which skills will remain viable as automation advances.

The antidote to career anxiety is separating transient implementation techniques from enduring foundational primitives. Throughout computing history, the introduction of higher levels of abstraction has never eliminated the demand for deep thinkers; it has simply elevated the frontier of human effort to higher levels of systemic complexity.

When compiled languages like C and Fortran replaced raw machine assembly, the world did not need fewer programmers—it needed millions more, because software could suddenly address problems of vastly greater scope and ambition. Generative AI represents another dramatic layer of abstraction atop the software stack.

When basic syntax and boilerplate generation become effortless commodities, career leverage shifts toward skills that cannot be resolved through statistical pattern matching alone:

1. **Problem Framing and Requirement Disambiguation:** Taking a messy, politically fraught, ambiguous business problem and distilling it into a precise, mathematically sound technical specification. AI can answer questions with remarkable speed, but it cannot tell you which problem is worth solving.

2. **Distributed Systems Thinking and Architecture:** Understanding how isolated services interact across high-latency networks, how data schemas evolve over a five-year horizon, where cascading failures occur, and how architectural coupling impacts team velocity.

3. **Adversarial Security and Diagnostic Rigor:** The diagnostic instinct to look at a working system and ask: *Where will this fail when the network partitions? How will a malicious attacker exploit this invariant? What edge cases did the training distribution completely ignore?*

4. **Domain Empathy and Human Alignment:** Understanding the unstated workflows, regulatory constraints, and daily frustrations of the human end-users for whom the software is built.

To build an antifragile career, cultivate a "T-shaped" capability: uncompromising, deep mastery of foundational principles (computer architecture, data structures, networking, systems design, and clear written communication) paired with broad dexterity across modern generative tools.

Keeping Ownership of Your Work

At the heart of all professional craftsmanship lies a simple, ancient principle: ownership. When your name is on the artifact, you are the author. You celebrate when it thrives, and you step forward to fix it when it breaks.

Generative artificial intelligence tempts us to dilute this relationship—to treat work as something that happens through us rather than from us. Resisting that temptation is the central ethical and practical challenge of the modern workplace.

To maintain uncompromising ownership, enforce *The Maintainer's Standard*: Never commit, deploy, or sign off on any piece of code, architectural design, or strategic document that you cannot explain line by line, defend under technical interrogation, and refactor by hand in real time without the assistance of an AI tool.

If an AI tool generates a complex regular expression, a distributed locking script, or a delicate CSS layout, and you paste it into your codebase without understanding every single mechanism, you have surrendered ownership. You have introduced an unvetted black box into a system you are paid to understand. You are no longer an engineer; you have become a caretaker of mysteries.

Maintain ownership through deliberate, daily engineering practices:

- **Write Test Harnesses First (TDD):** Before asking an AI model to implement a complex function, write the unit tests, boundary conditions, and failure cases yourself. Defining the boundaries beforehand forces you to understand the problem deeply before the machine generates a line of code.
- **Perform Line-by-Line Diff Audits:** Treat machine output like code submitted by a brilliant but reckless intern. Read every line slowly. Question variable names, check boundary allocations, verify concurrency safety, and eliminate unnecessary abstractions.
- **Refactor in Your Own Hand:** If a generated snippet feels dense or uses unfamiliar idioms, take the time to type it out yourself, refactoring it into patterns that align cleanly with your team's architectural standards.

Ownership is not a burden to be avoided through automation. It is the primary source of meaning, pride, and mastery in professional life. The satisfaction of being a true craftsman comes from knowing that you took a difficult problem, applied your judgment, and built a resilient solution that genuinely works.

Key Takeaways

- **Throughput Is Not Competence:** Fast generation of unvetted artifacts creates brittle architectures and erodes foundational diagnostic intuition.
- **Practice Collaborative Mastery:** Reject the passive replacement mindset; use AI as an assistant to explore options while keeping architectural intent and validation in human hands.
- **Defend Team Trust:** Enforce radical transparency, heightened review scrutiny for synthetic code, and absolute personal accountability for all deliverables.
- **Invest in Timeless Fundamentals:** As syntax is commoditized, career leverage belongs to those who master problem framing, distributed systems thinking, and adversarial diagnostics.
- **The Maintainer's Standard:** Never ship, merge, or sign off on any artifact that you cannot explain, defend, and refactor by hand without tool assistance.

Try This

▷ **PRACTICAL EXERCISE**

Review the last three code commits or documents you authored with the assistance of an AI tool. Perform a strict "Maintainer's Audit":
Print out the diffs or view them side by side on a clean screen. For every generated function, configuration block, or argumentative paragraph, ask yourself: *Can I explain the exact mechanics of this line from memory? Do I understand every failure mode? Could I rewrite this from scratch on a whiteboard during a live interview?*
If you find a section that you cannot explain without consulting the AI tool, delete it immediately. Spend thirty minutes studying the underlying documentation and rewrite the implementation entirely by hand.

Important Information

◇ **CONTEXT & GUIDELINES**

In sociotechnical research and human factors engineering, *Ironies of Automation* (a seminal 1983 paper by psychologist Lisanne Bainbridge) demonstrates that as automated systems become more capable at handling routine operations, human operators are left with two primary tasks: monitoring the automation (which human attention is notoriously ill-equipped to do for prolonged periods) and intervening during rare, catastrophic edge cases that the automation cannot resolve. Crucially, because the automation handles all routine tasks, the human operator never gets the daily practice required to maintain the diagnostic skills needed during those critical emergencies.
In software engineering and knowledge work, relying on AI for routine coding risks creating this exact automation irony: engineers who cannot debug catastrophic production failures because they never practice routine mechanics by hand.

In the Next Chapter

Technical craftsmanship requires personal ownership; artistic and creative work demands something even deeper. Chapter Seven explores what it means to write, design, and make original things in a world filled with instant synthetic output.

Reflection Questions

◇

- How has your reliance on AI tools affected your confidence in diagnosing complex production failures under pressure?

- Does your team currently maintain explicit, open norms regarding AI usage in code reviews and architectural proposals, or is it an unspoken grey zone?
- When reviewing a colleague's machine-assisted pull request, do you review it with more scrutiny or less scrutiny than human-authored code?
- Which foundational technical skills in your engineering stack do you feel are most at risk of atrophying through over-reliance on generative assistants?
- What concrete steps can you take this month to ensure that your career leverage is rooted in timeless systems thinking rather than transient prompt interfaces?